Analizador léxico y sintáctico del grupo 24

Informe de reentrega de TP1 y TP2. Esta versión documenta las correcciones del compilador entregado y reemplaza las descripciones técnicas desactualizadas del informe anterior. El informe anterior no llegó al docente por un error en la entrega; esa omisión se subsana incluyendo este documento junto con el código y las pruebas.

Integrantes: Rodrigo Ruben Bejarano, rodrigobejarano456@gmail.com; Viggo Lombardi, viggolombardi@gmail.com.

Temas asignados: 3, 7, 10, 14, 15, 18, 19, 22, 24, 29, 32 y 34.

1 Introducción y alcance

El compilador está escrito en C. El analizador léxico reconoce tokens mediante una matriz de transición de estados y otra de acciones semánticas. Bison genera el analizador sintáctico a partir de grammar.y. La tabla de símbolos almacena identificadores, constantes y cadenas de caracteres mediante nodos dinámicos.

Esta entrega reconoce estructuras, informa errores con su línea y continúa cuando existe un punto de recuperación. No construye un árbol sintáctico explícito ni genera código. La comprobación de tipos, la evaluación de comptime, la compatibilidad estructural entre clases y la desambiguación de herencia del tema 32 corresponden a las etapas siguientes.

Los nombres, comentarios y mensajes del código se mantienen en inglés. Este informe explica en español las decisiones y sus fundamentos.

2 Requisitos de los temas asignados

Tema	Requisito implementado
3	INTEGER de 16 bits, sufijo $i, rango entre -32768 y 32767; el parser resuelve el signo.
7	SINGLEF con punto y dígitos decimales obligatorios; parte entera y exponente s opcionales.
10	Cadenas multilínea entre comillas dobles; la TS conserva su contenido sin saltos de línea.
14	from ID := constante to constante by constante repeat cuerpo; cuerpo simple o bloque.
15	Comentarios de línea iniciados con //; terminan al final de línea o de archivo.
18	Asignaciones con = dentro de expresiones, con identificador o constante como valor asignado.
19	Llamadas con argumentos y orden obligatorio de evaluación entre corchetes.
22	Declaraciones comptime tipo lista_de_variables;.
24	Declaraciones de clases con nombre y código obligatorio; atributos y métodos.
29	Referencias posicionales objeto[constante] u objeto[identificador].
32	Declaraciones extends con una lista de clases base.
34	Conversión explícita toi(expr), admitida como factor de una expresión.

Las expresiones no permiten paréntesis de agrupación general. Los paréntesis de condiciones, llamadas, pout, ret y toi forman parte de esas construcciones y sí están admitidos.

3 Analizador léxico

3.1 Estados y columnas

El autómata conserva 15 estados y 17 columnas. get_col asigna a cada carácter una columna. Las columnas i y s distinguen el sufijo entero y el marcador de exponente; dentro de un identificador ambas siguen comportándose como letras.

El estado 0 inicia tokens. El estado 1 reconoce identificadores y palabras reservadas; 2 y 3 distinguen división y comentario; 4 reconoce := y !=; 5 y 6 reconocen enteros; 7 a 11 reconocen flotantes; 12 reconoce cadenas; 13 distingue = y ==; 14 reconoce comparadores con < o >.

En cada iteración se ejecuta sem_act_mat[state][column] y se consulta transition_table[state][column]. La acción puede devolver -1 para continuar, un token positivo o YYerror. El -1 es interno: yylex nunca lo devuelve como error al parser, porque Bison interpreta los resultados no positivos como fin de entrada.

F_CONS indica un final que consume el carácter actual. F_RET indica un final que devuelve ese carácter al archivo para el token siguiente. E indica una transición inválida. Una acción de validación puede rechazar un lexema aunque su transición sea de finalización.

La matriz conserva 0 -> 5 con el primer dígito y 5 -> 5 con los siguientes. Los anexos se generan directamente desde los dos archivos C; los arcos se rotulan con el nombre real de la acción, no con una numeración independiente. El anexo A contiene ambas matrices y el anexo B muestra todas las transiciones agrupadas por estado, destino y acción. lexer.dot permite representar el autómata completo con Graphviz.

3.2 Fin de archivo y líneas

read_input_char sustituye el primer EOF por un salto de línea artificial. Ese único delimitador permite finalizar tokens que necesitan un carácter posterior para determinar su final. La variable is_synthetic identifica ese carácter: update_line_number no incrementa la línea y push_back_char no lo devuelve al archivo. La lectura posterior devuelve el EOF real.

El mecanismo no agrega un cierre de cadena ni completa constantes mal formadas. Una cadena abierta al EOF produce Unclosed string literal. Los comentarios // pueden finalizar al EOF sin error. El lexer cuenta CRLF como un solo salto de línea.

3.3 Acciones semánticas

Acción	Responsabilidad
sa_init	Reinicia el lexema y agrega su primer carácter.
sa_append	Agrega el carácter; omite CR y LF del contenido de las cadenas.
sa_token_consume	Devuelve un token de un carácter consumiendo el carácter leído.
sa_token_buffered	Devuelve el carácter almacenado; la transición F_RET preserva el siguiente.
sa_ignore	No genera un token ni modifica el lexema.
sa_identifier	Reconoce reservadas, valida identificadores, trunca los largos y registra los válidos.

Acción	Responsabilidad
sa_int_const	Completa $i, convierte con strtol y valida la magnitud antes de registrar.
sa_float_const	Convierte una copia con strtod y valida el rango abierto de SINGLEF.
sa_init_chain	Prepara el búfer vacío para una cadena entre comillas.
sa_chain	Registra una cadena que tiene comilla de cierre y devuelve TOKEN_CHAIN.
sa_multi_char_op	Reconoce solamente :=, >=, <=, == y !=.
sa_dot_or_float_error	Devuelve un punto aislado o rechaza un flotante sin dígitos decimales.
sa_float_decimal_error	Rechaza un intento de exponente sin punto y parte decimal.
sa_integer_literal_error	Informa ausencia o escritura incorrecta del sufijo $i.
sa_float_exponent_error	Rechaza exponentes sin dígitos después de s o del signo.
sa_error	Informa caracteres o secuencias generales inválidos, sin reconstruir contextos numéricos.

sa_float_decimal_error se asocia a la transición del estado 5 con s. sa_dot_or_float_error se asocia al estado 7. De esta manera 12s10 y 12. tienen diagnósticos específicos, y sa_error deja de repetir validaciones de constantes.

3.4 Identificadores y operadores

Las palabras reservadas se comparan sin distinguir mayúsculas, como exige TP1. Un identificador con mayúsculas se rechaza: no se transforma en otro nombre ni se registra en la TS. Un identificador válido de más de 22 caracteres se trunca e informa un warning, tal como establece el enunciado.

En el estado 13, < y > finalizan el token = mediante F_RET y sa_token_buffered. Por eso => produce = y >, y =< produce = y <. El parser diagnostica la secuencia incorrecta. No se devuelve un comparador válido distinto del texto escrito.

4 Tabla de símbolos y constantes

4.1 Estructuras y propiedad de memoria

hash_map_t contiene un arreglo de 256 buckets y su capacidad. Cada bucket encabeza una lista simplemente enlazada de map_node_t. El número de buckets es fijo, pero el número de símbolos no: cada nodo, lexema y categoría se reserva dinámicamente. Las colisiones se resuelven recorriendo y ampliando la lista correspondiente.

La función djb2 calcula el hash del lexema. map_put reutiliza una entrada cuando coinciden key y value, o agrega un nodo a la lista. Así el mismo texto puede existir como identificador y como cadena sin confundir sus categorías. La búsqueda tiene costo esperado cercano a constante cuando las listas son cortas; el peor caso es lineal en la lista de colisiones.

key guarda el lexema y value guarda la categoría léxica: ID, INTEGER, SINGLEF o CHAIN. La columna Type de la salida representa esa categoría, no el tipo declarado de una variable. CHAIN es una cadena literal; el lenguaje no incorpora un tipo declarado String. Los char* de C son una representación de memoria y no un tipo del lenguaje compilado.

Para identificadores, constantes y cadenas válidos, yylval.symbol_ref devuelve la referencia estable al nodo. La TS es propietaria de esos nodos; los valores de Bison los toman prestados. No se liberan mediante un destructor de token. map_free libera toda la tabla después de finalizar el parser. Las reservas de memoria se comprueban antes de usar sus resultados.

4.2 Validación antes del alta

Los identificadores con mayúsculas y las constantes fuera de rango no se insertan ni se sustituyen por valores válidos. Tampoco se crean categorías INVALID_INTEGER o INVALID_SINGLEF en la TS. El lexer informa el error y devuelve YYerror para iniciar recuperación.

Para enteros, strtol controla el desbordamiento de la conversión y el sufijo completo. Las magnitudes mayores que 32768 se rechazan incluso si el texto contiene cientos de dígitos. Para SINGLEF se reemplaza s por e solamente en una copia de trabajo, se comprueba strtod, su indicador de rango y el consumo completo del texto.

El rango flotante usa desigualdades estrictas: 1.17549435s-38 < abs(x) < 3.40282347s+38, o x = 0.0. Los extremos se rechazan; no se confunden con valores interiores ni con subnormales admitidos por una representación de C.

4.3 Signo y límite INTEGER

TP2 exige resolver el signo en el parser. El lexer acepta provisionalmente la magnitud 32768 para permitir -32768$i. El nodo se marca provisional y conserva su referencia durante todo el análisis.

La regla constant rechaza el uso positivo de ese nodo. En el uso negativo registra -32768$i como INTEGER válido. También registra las demás constantes negativas usando el signo y la categoría del token. La magnitud léxica de un flotante ya determina su validez porque el rango es simétrico.

map_purge_provisional elimina los nodos provisionales después de yyparse y antes de imprimir la TS. Nunca los libera mientras puedan estar referenciados por la pila o por el token pendiente. La TS final no contiene 32768$i positivo, aunque una referencia provisional haya sido necesaria para cumplir TP2. Si ese límite se descarta durante recuperación también se elimina al terminar.

5 Analizador sintáctico

5.1 Construcciones válidas

La gramática conserva expr para suma y resta y term para multiplicación y división. La recursión izquierda fija la asociatividad y la estructura expr/term/factor fija la precedencia sin agregar paréntesis de agrupación ni precedencias artificiales para ocultar conflictos.

assign reconoce asignaciones a identificadores con := y a attr_ref con := o =. attr_access se elimina porque separaba innecesariamente las asignaciones y sus recuperaciones. Se conserva = para atributos por los ejemplos del tema 29 y := por la alternativa reconocida como correcta en la devolución. No se habilita = como asignación general de sentencia a un identificador.

assign_chain conserva el tema 18: sus valores asignados son identificadores o constantes. Las invocaciones y referencias posicionales pueden intervenir como operandos normales, pero no amplían ese valor asignado. Las llamadas a funciones y métodos exigen la lista de constantes de orden del tema 19. La comprobación semántica de ese orden se deja para las etapas siguientes.

Los bucles conservan la terminación definida para esta entrega. Un cuerpo simple tiene su punto y coma y el bucle tiene otro: repeat a := 1$i;;. Un cuerpo begin/end contiene sentencias terminadas en ; y se escribe end; para terminar el bucle. Este criterio está documentado y tiene pruebas válidas e inválidas específicas.

5.2 Desarrollo y resolución de conflictos

La gramática parte de la versión entregada. Primero se unificaron las asignaciones de atributos y se agregaron producciones específicas de operandos faltantes. Luego se separaron los encabezados de funciones, métodos y bucles de sus cuerpos.

Esta separación resuelve una dificultad de recuperación: un error en el encabezado no debe quedar pendiente hasta después de analizar un cuerpo que puede contener otros errores independientes. Los encabezados se diagnostican antes de entrar al cuerpo y cada error conserva su origen hasta el punto de recuperación apropiado.

La generación del parser se ejecuta con -Wall, -Werror=conflicts-sr y -Werror=conflicts-rr. La gramática resultante genera cero conflictos de ambos tipos. No se usa %expect para ocultarlos. Para investigar un conflicto futuro se puede usar el informe de estados y -Wcounterexamples con Bison 3.7 o posterior.

No se afirma que los delimitadores por sí solos garanticen ausencia de conflictos: el criterio se comprueba al generar el parser. Las producciones de diagnóstico reconocen formas incorrectas, emiten errores y nunca convierten su reconocimiento en compilación exitosa.

5.3 No terminales

No terminal	Función
statements	Programa completo y recuperación de delimitadores principales.
program_name	Identificador de programa; informa su ausencia.
decl_list	Lista no vacía de declaraciones de variables, clases y funciones.
var_decl	Variables normales o comptime y recuperación de declaraciones.
id_list	Identificadores separados por comas y errores de separadores.
statement	Cuerpo ejecutable del programa.
compound_stmt	Secuencia no vacía de sentencias; detecta terminadores ausentes.
simple_stmt	Asignación, selección, bucle, salida o retorno y recuperación por sentencia.
single_stmt	Una sentencia usada como cuerpo simple.
type	Tipo primitivo o nombre de clase.
primitive_type	INTEGER o SINGLEF.
assign	Todas las asignaciones de sentencia, incluidas las posicionales.
expr	Sumas, restas y sus diagnósticos de operandos.
term	Productos, divisiones y sus diagnósticos de operandos.
constant	Constantes numéricas, signo y validación del límite positivo INTEGER.

No terminal	Función
assign_chain	Prefijo de asignaciones con = del tema 18.
factor	Factor normal o asignación interna del tema 18.
factor_base	Llamada, identificador, constante, toi o atributo posicional.
call	Invocación a función o método con orden obligatorio.
arg_list	Expresiones usadas como argumentos.
const_list	Constantes de la lista de orden.
cond	Comparación entre expresiones y diagnósticos de comparadores u operandos.
stmt_block	Bloque begin/end o una sentencia.
if_stmt	Selección con condición, else opcional y end_if.
else_stmt	Rama alternativa de la selección.
for_loop	Bucle completo con cuerpo y terminador.
for_header	Encabezado del tema 14 y errores antes de entrar al cuerpo.
func_def	Definición de función con cuerpo y declaraciones locales opcionales.
function_header	Tipo, nombre y parámetros de función.
function_scope	Guarda de contexto para permitir ret dentro de funciones y métodos.
param_decl_list	Parámetros formales con tipo y nombre.
method_def	Definición de método dentro de una clase.
method_header	Tipo, nombre y parámetros de método.
class_def	Clase con nombre, código y miembros.
class_body	Lista de miembros de clase.
extends_stmt	Sentencia extends del tema 32.
extends_id_list	Nombres de clases base separados por comas.
class_member	Atributo, método o sentencia extends.
attr_ref	Referencia posicional a atributo del tema 29.
pout_stmt	Salida de una expresión o cadena.
ret_stmt	Retorno con expresión obligatoria y validación del contexto.

6 Errores y recuperación

6.1 Errores léxicos

Se consideran caracteres inválidos, identificadores con mayúsculas, enteros fuera de rango o con sufijo incorrecto, flotantes fuera de rango, parte decimal ausente, exponente incompleto y cadenas sin cierre. Los identificadores largos producen advertencias y se truncan.

Ante un lexema inválido el lexer lo descarta. Si el carácter de cierre es puntuación válida, lo preserva para el parser. En una secuencia inválida consume hasta un delimitador de recuperación. Los errores léxicos se informan una sola vez y YYerror inicia la recuperación sintáctica sin agregar automáticamente un mensaje genérico.

6.2 Errores sintácticos

Se consideran los componentes omitidos del programa, separadores de declaraciones y sentencias, parámetros incompletos, retorno fuera de función o sin expresión, argumentos u orden ausentes, condiciones incompletas, comparadores invertidos, componentes omitidos de bucles, código de clase ausente, extends incompleto, uso incorrecto de := en expresiones, cadenas en operaciones y toi sin expresión.

Hay diagnósticos específicos para operandos derechos de +, -, * y / y para operandos izquierdos ausentes. El signo - seguido de una constante numérica es válido. Un - sin constante o aplicado a un identificador no incorpora una negación general al lenguaje.

Las producciones con error recuperan usando los delimitadores de la construcción: ; para sentencias y declaraciones,) para argumentos y conversiones, y las palabras del encabezado para bucles. Los delimitadores externos no se descartan si todavía permiten completar la construcción que los contiene.

RECOVER usa yyerrok al terminar una recuperación local. Usa yyclearin únicamente si queda el marcador YYUNDEF introducido por un YYerror léxico; no descarta un ;,) o] real. %locations conserva la ubicación de los tokens y parse.lac full mejora la exactitud de los tokens esperados.

6.3 Origen y diagnóstico pendiente

diagnostics.c mantiene el origen léxico o sintáctico del episodio actual. El número de línea sirve para ubicar el mensaje, no para decidir qué errores ocultar. Un error léxico y un pout() inválido en la misma línea producen dos mensajes.

yyreport_syntax_error registra y cuenta el error al detectarlo. La producción que logra recuperar puede especializar su mensaje. Si el parser termina antes de que esa producción reduzca, finish_error_recovery imprime el contexto pendiente. Por eso un archivo con solo el nombre del programa se rechaza con un diagnóstico y no termina con cero errores.

finish_statement cierra los episodios ya diagnosticados, pero conserva un contexto todavía pendiente de una construcción externa. Los encabezados de funciones y bucles informan sus errores antes del cuerpo. El destructor de function_scope restaura la profundidad si Bison descarta una función durante recuperación; al finalizar normalmente se desactiva la guarda antes de decrementar.

La compilación solo es exitosa si yyparse retorna cero y global_errors también es cero. En EOF sin cierre de función se informa el problema y se termina el parser porque ya no queda entrada que recuperar. El programa libera sus recursos y devuelve 1 si hubo errores.

7 Validación

La suite contiene 88 archivos con expectativas explícitas. run_tests.sh compila y ejecuta scripts/run_tests.py; el verificador compara código de salida, cantidad y texto de los diagnósticos, ubicaciones, advertencias, estructuras reconocidas, secuencias de tokens y símbolos presentes o ausentes. Si una expectativa falla, la suite devuelve 1.

Las 62 pruebas entregadas se conservan. Se actualizan dos expectativas por los cambios de comportamiento: el comparador invertido ahora produce un error sintáctico, y la falta de terminador de bucle se identifica como tal. Las pruebas adicionales cubren operandos de ambos lados, distintos contextos de expresión, eliminación de constantes inválidas, desbordamiento de enteros grandes, límites con signo, errores independientes en una línea, encabezados y cuerpos con errores, EOF sin salto final, CRLF y cadenas multilínea.

La validación incluye los casos válidos combinados de todos los temas asignados. El criterio exige tanto rechazar entradas incorrectas como preservar las correctas. El parser se genera sin conflictos y el ejecutable se comprueba además con AddressSanitizer, UndefinedBehaviorSanitizer y detección de fugas de memoria.

Para reproducir la comprobación principal: ./run_tests.sh. Para ver las salidas completas: ./run_tests.sh --verbose. El resultado registrado para esta revisión es 88/88 pruebas aprobadas.

8 Uso de inteligencia artificial

El informe original atribuye a Gemini la matriz de acciones y los casos de prueba, a Claude la adaptación de la gramática a Yacc/Bison y a IA la implementación inicial de la tabla hash. Esa atribución se conserva como antecedente; no se inventan modelos, prompts ni intervenciones históricas que no constan en el material disponible.

La revisión actual utilizó Codex de OpenAI para comparar versiones, analizar la devolución, consultar la teoría y Bison, elaborar el plan e implementar las correcciones. Su intervención comprende lexer, acciones, TS, diagnósticos, gramática, runner, pruebas y documentación. docs/ai_usage.md reproduce las instrucciones disponibles y delimita las atribuciones históricas.

La aceptación de una propuesta generada por IA se basa en su correspondencia con los enunciados y en pruebas ejecutables. El equipo debe poder explicar el autómata, los registros provisionales, los préstamos de referencias a Bison, los puntos de sincronización y las producciones de diagnóstico durante la defensa.

9 Conclusiones

La revisión conserva el diseño por matrices y corrige el límite entre reconocimiento y recuperación. Los lexemas inválidos dejan de generar símbolos o tokens válidos de reemplazo. Los comparadores invertidos conservan sus caracteres originales, las asignaciones posicionales comparten una regla y las operaciones tienen diagnósticos específicos de operandos.

El informe y las pruebas describen el comportamiento real del código. La recuperación permite continuar sin confundir finalización del parser con aceptación del programa. Las etapas semánticas y de generación de código mantienen el alcance previsto para los trabajos siguientes.

10 Referencias

Material de la cátedra: Tp01-2026.pdf; Tp02-2026.pdf; 03.Análisis Léxico.pdf; 05.Parsing Ascendente.pdf; 07.1.Análisis Sintáctico-Yacc.pdf, disponibles en la carpeta teoria compiladores.

GNU Bison, Error Recovery: https://www.gnu.org/software/bison/manual/html_node/Error-Recovery.html

GNU Bison, Special Tokens: https://www.gnu.org/software/bison/manual/html_node/Special-Tokens.html

GNU Bison, Syntax Error Reporting Function:
https://www.gnu.org/software/bison/manual/html_node/Syntax-Error-Reporting-Function.html

GNU Bison, LAC: https://www.gnu.org/software/bison/manual/html_node/LAC.html

GNU Bison, Freeing Discarded Symbols:
https://www.gnu.org/software/bison/manual/html_node/Destructor-Decl.html

GNU Bison, Counterexamples:
https://www.gnu.org/software/bison/manual/html_node/Counterexamples.html

Anexo A Matrices del analizador léxico

Destinos de transition_table, extraídos del código C. D es dígito; L es letra excepto i y s; nl incluye CR y LF; ws incluye espacio y tabulación; op*() incluye *, (,), ;, coma y corchetes.

Estado	L	i	s	D	_	.	$	+-	/	op*()	=	<>	!:	"	nl	ws	other
0	1	1	1	5	E	7	E	F_CONS	2	F_CONS	13	14	4	12	0	0	E
1	1	1	1	1	1	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET
2	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	3	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET
3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	0	3	3
4	E	E	E	E	E	E	E	E	E	E	F_CONS	E	E	E	E	E	E
5	E	E	E	5	E	7	6	E	E	E	E	E	E	E	E	E	E
6	E	F_CONS	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E
7	F_RET	F_RET	F_RET	8	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET
8	F_RET	F_RET	9	8	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET
9	E	E	E	11	E	E	E	10	E	E	E	E	E	E	E	E	E
10	E	E	E	11	E	E	E	E	E	E	E	E	E	E	E	E	E
11	F_RET	F_RET	F_RET	11	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET
12	12	12	12	12	12	12	12	12	12	12	12	12	12	F_CONS	12	12	12
13	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_CONS	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET
14	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET	F_CONS	F_RET	F_RET	F_RET	F_RET	F_RET	F_RET

Anexo A Acciones para columnas 0 a 5

Nombres exactos de las funciones de sem_act_mat. Cada fila corresponde al estado actual. Las columnas conservan el orden de get_col.

Estado	L	i	s	D	_	.
0	sa_init	sa_init	sa_init	sa_init	sa_error	sa_init
1	sa_append	sa_append	sa_append	sa_append	sa_append	sa_identifier
2	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered
3	sa_ignore	sa_ignore	sa_ignore	sa_ignore	sa_ignore	sa_ignore
4	sa_error	sa_error	sa_error	sa_error	sa_error	sa_error
5	sa_integer_literal_error	sa_integer_literal_error	sa_float_decimal_error	sa_append	sa_integer_literal_error	sa_append
6	sa_integer_literal_error	sa_int_const	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error
7	sa_dot_or_float_error	sa_dot_or_float_error	sa_dot_or_float_error	sa_append	sa_dot_or_float_error	sa_dot_or_float_error
8	sa_float_const	sa_float_const	sa_append	sa_append	sa_float_const	sa_float_const
9	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_append	sa_float_exponent_error	sa_float_exponent_error
10	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_append	sa_float_exponent_error	sa_float_exponent_error
11	sa_float_const	sa_float_const	sa_float_const	sa_append	sa_float_const	sa_float_const
12	sa_append	sa_append	sa_append	sa_append	sa_append	sa_append
13	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered
14	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered

Anexo A Acciones para columnas 6 a 11

Nombres exactos de las funciones de sem_act_mat. Cada fila corresponde al estado actual. Las columnas conservan el orden de get_col.

Estado	$	+-	/	op*()	=	<>
0	sa_error	sa_token_consume	sa_init	sa_token_consume	sa_init	sa_init
1	sa_identifier	sa_identifier	sa_identifier	sa_identifier	sa_identifier	sa_identifier
2	sa_token_buffered	sa_token_buffered	sa_ignore	sa_token_buffered	sa_token_buffered	sa_token_buffered
3	sa_ignore	sa_ignore	sa_ignore	sa_ignore	sa_ignore	sa_ignore
4	sa_error	sa_error	sa_error	sa_error	sa_multi_char_op	sa_error
5	sa_append	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error
6	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error
7	sa_dot_or_float_error	sa_dot_or_float_error	sa_dot_or_float_error	sa_dot_or_float_error	sa_dot_or_float_error	sa_dot_or_float_error
8	sa_float_const	sa_float_const	sa_float_const	sa_float_const	sa_float_const	sa_float_const
9	sa_float_exponent_error	sa_append	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error
10	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error
11	sa_float_const	sa_float_const	sa_float_const	sa_float_const	sa_float_const	sa_float_const
12	sa_append	sa_append	sa_append	sa_append	sa_append	sa_append
13	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_multi_char_op	sa_token_buffered
14	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_multi_char_op	sa_token_buffered

Anexo A Acciones para columnas 12 a 16

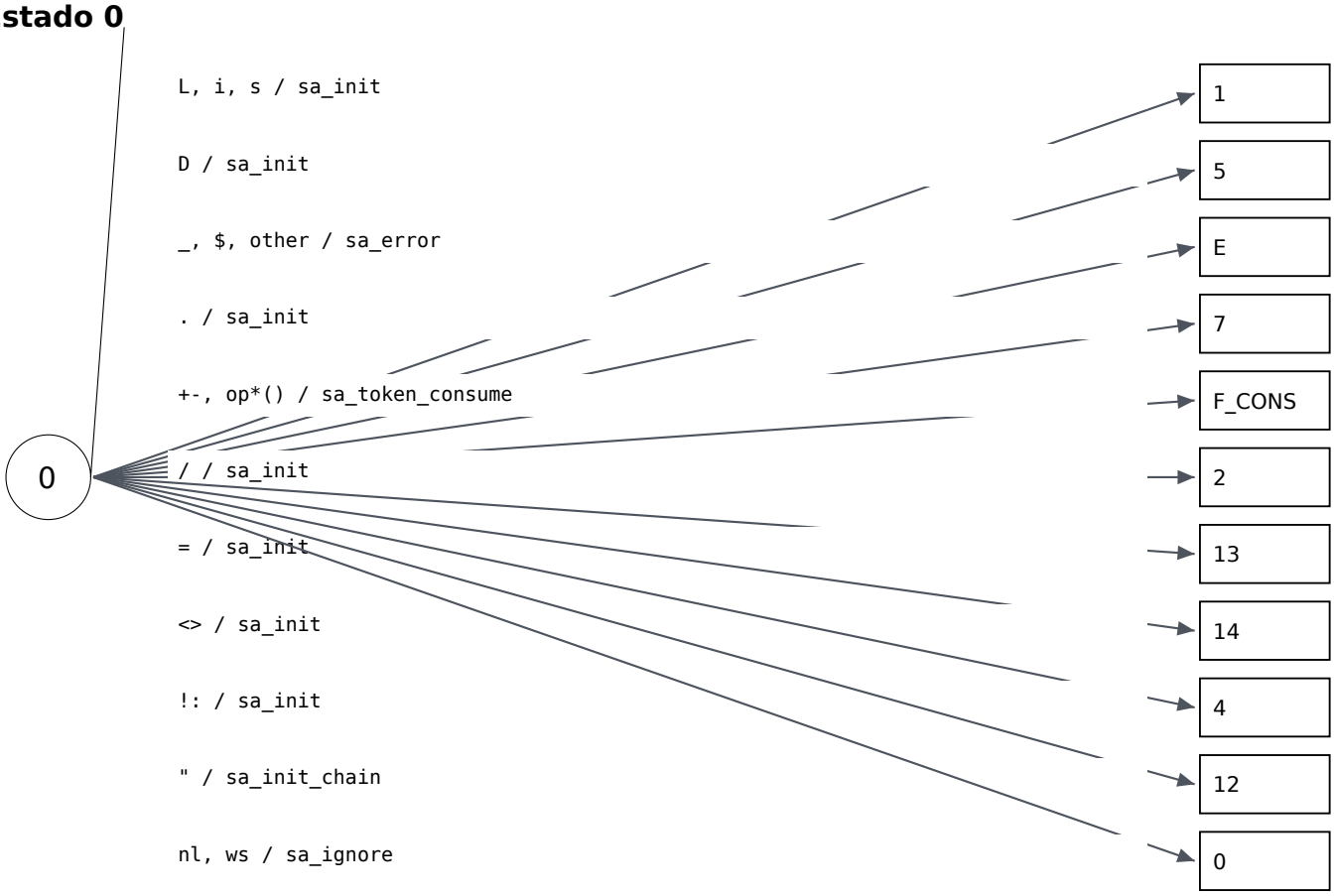
Nombres exactos de las funciones de sem_act_mat. Cada fila corresponde al estado actual. Las columnas conservan el orden de get_col.

Estado	!:	"	nl	ws	other
0	sa_init	sa_init_chain	sa_ignore	sa_ignore	sa_error
1	sa_identifier	sa_identifier	sa_identifier	sa_identifier	sa_identifier
2	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered
3	sa_ignore	sa_ignore	sa_ignore	sa_ignore	sa_ignore
4	sa_error	sa_error	sa_error	sa_error	sa_error
5	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error
6	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error	sa_integer_literal_error
7	sa_dot_or_float_error	sa_dot_or_float_error	sa_dot_or_float_error	sa_dot_or_float_error	sa_dot_or_float_error
8	sa_float_const	sa_float_const	sa_float_const	sa_float_const	sa_float_const
9	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error
10	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error	sa_float_exponent_error
11	sa_float_const	sa_float_const	sa_float_const	sa_float_const	sa_float_const
12	sa_append	sa_chain	sa_append	sa_append	sa_append
13	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered
14	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered	sa_token_buffered

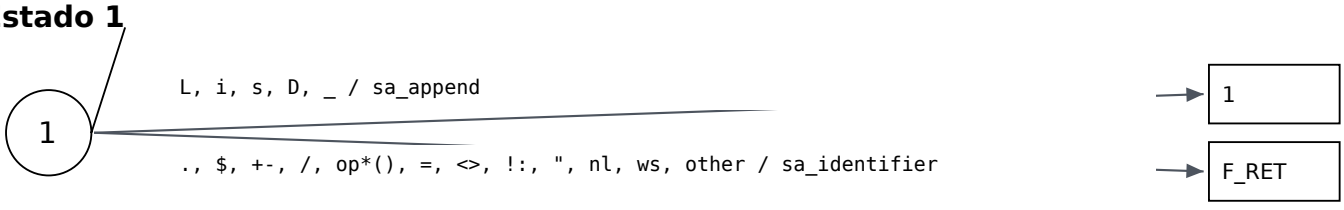
Anexo B Diagramas de transición

Cada nodo de origen representa un estado. Los destinos con el mismo nombre representan el mismo estado del autómata, aunque se repitan en estos diagramas locales. Cada arco agrupa columnas que tienen el mismo destino y la misma acción. La tabla del anexo A conserva las 255 celdas individuales.

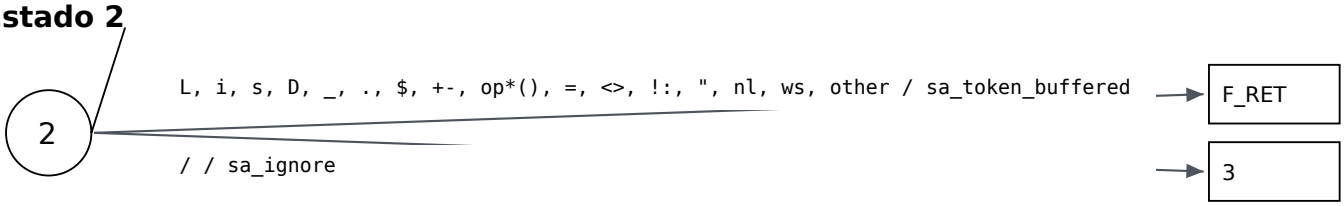
Estado 0



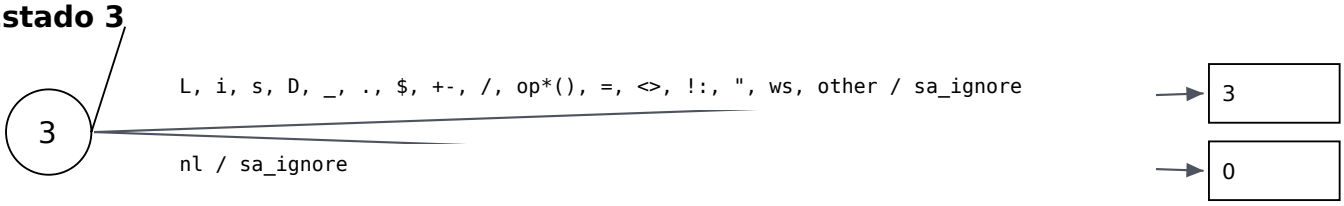
Estado 1



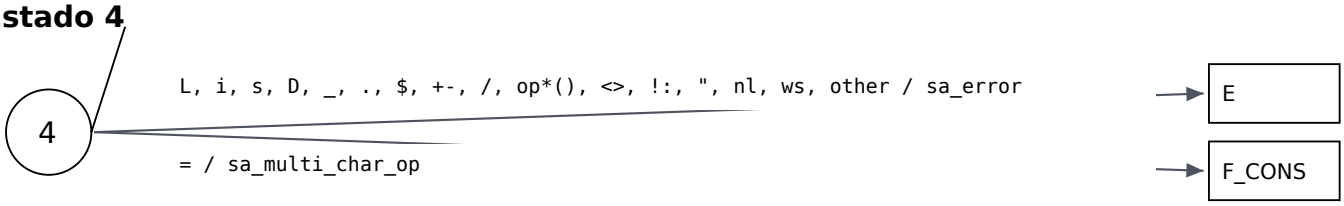
Estado 2



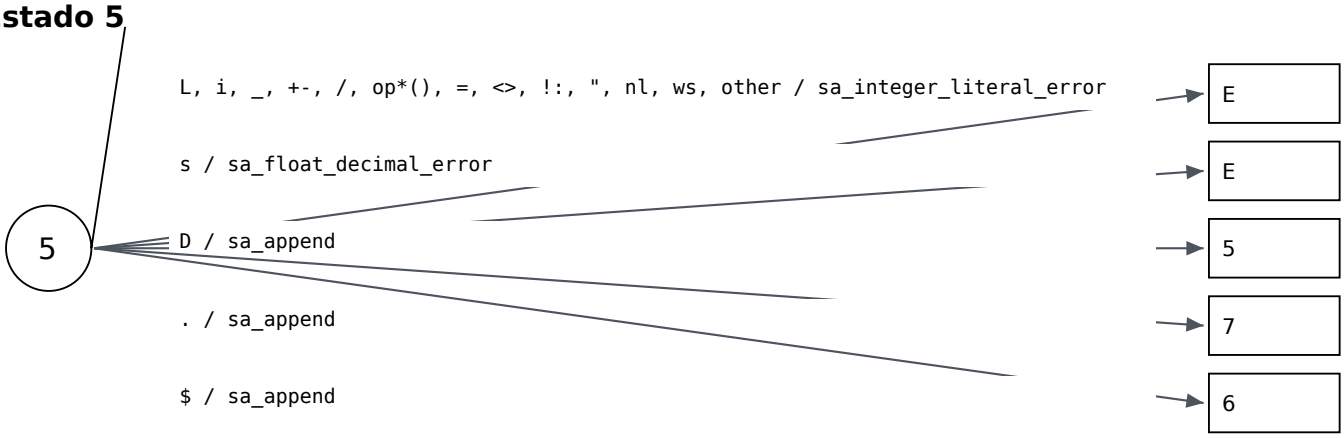
Estado 3



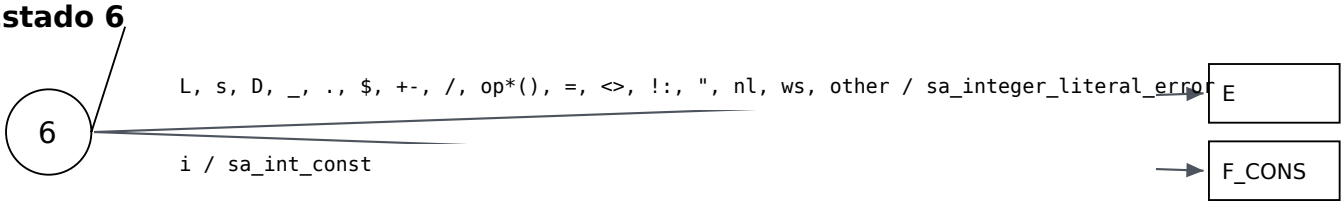
Estado 4



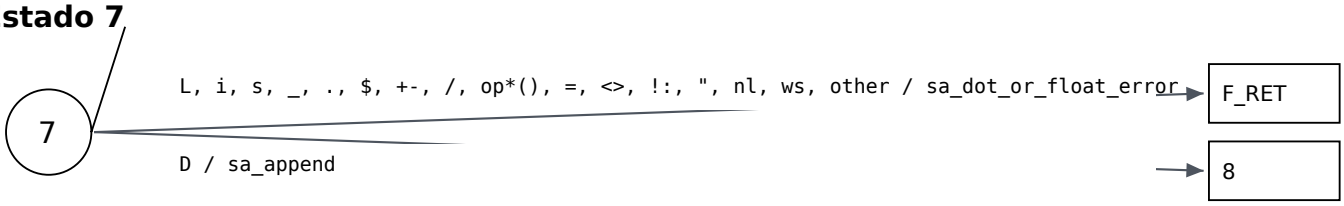
Estado 5



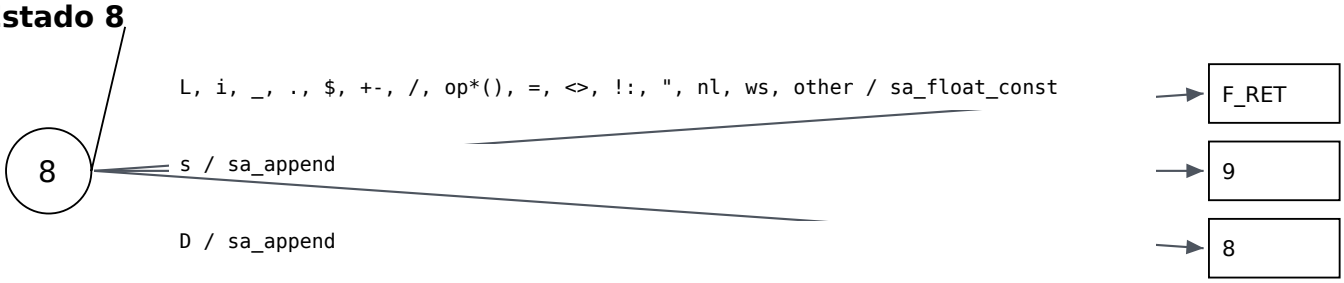
Estado 6



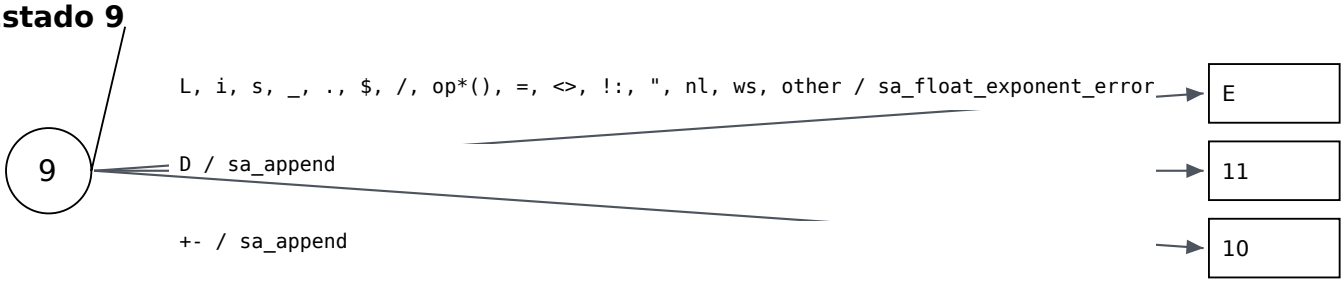
Estado 7



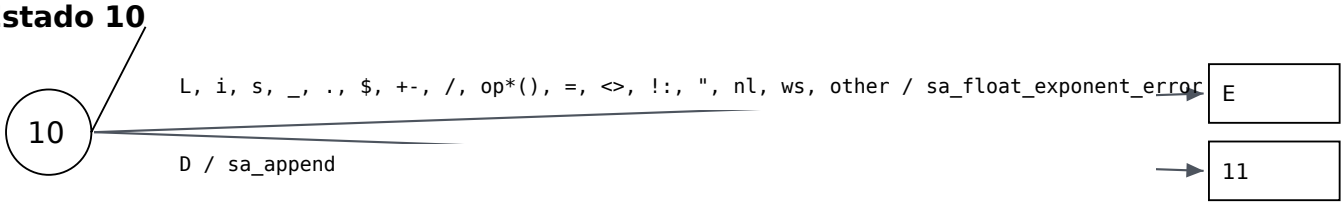
Estado 8



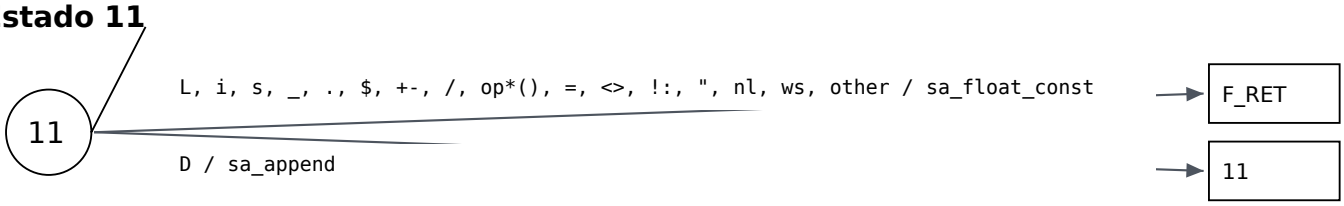
Estado 9



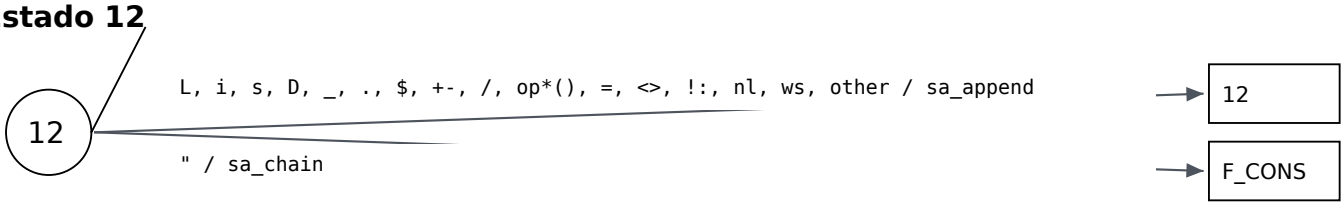
Estado 10



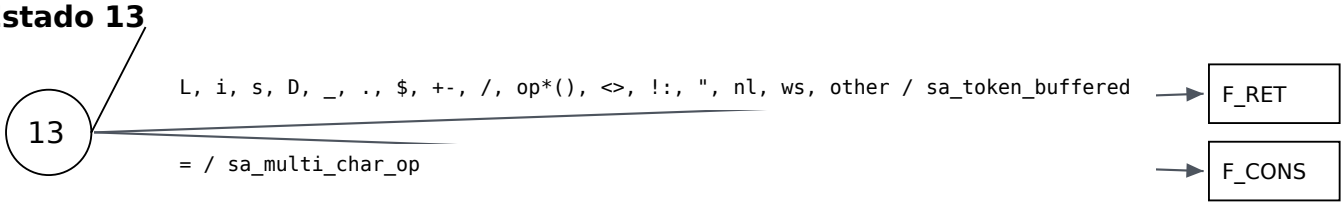
Estado 11



Estado 12



Estado 13



Estado 14

